

深度学习基础理论

什么是神经网络？

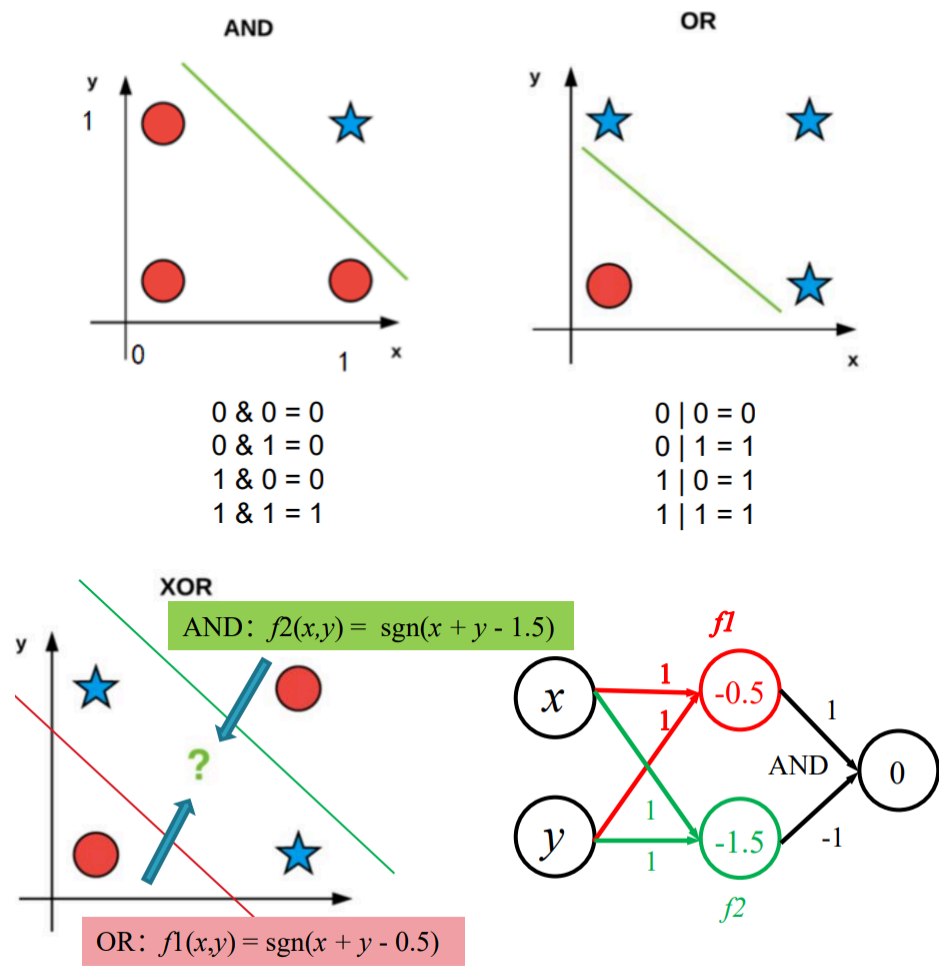
神经网络是一种由多层神经元组成的非线性函数模型，通过学习输入与输出之间的映射关系，实现对复杂数据的建模与预测。学习的本质是逐步压缩数据的过程。

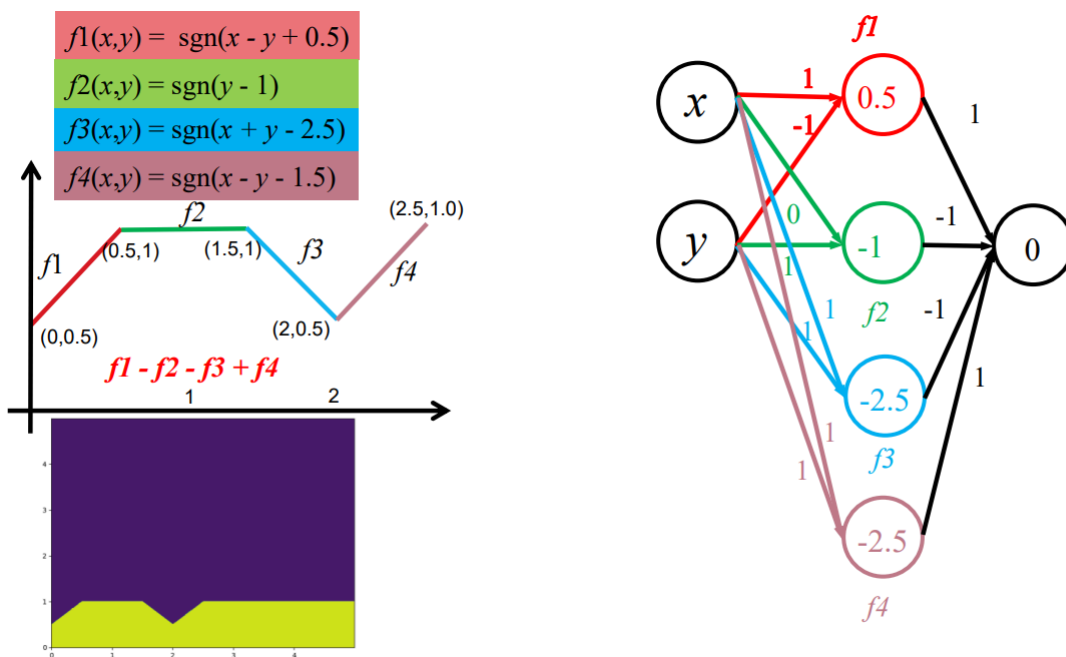
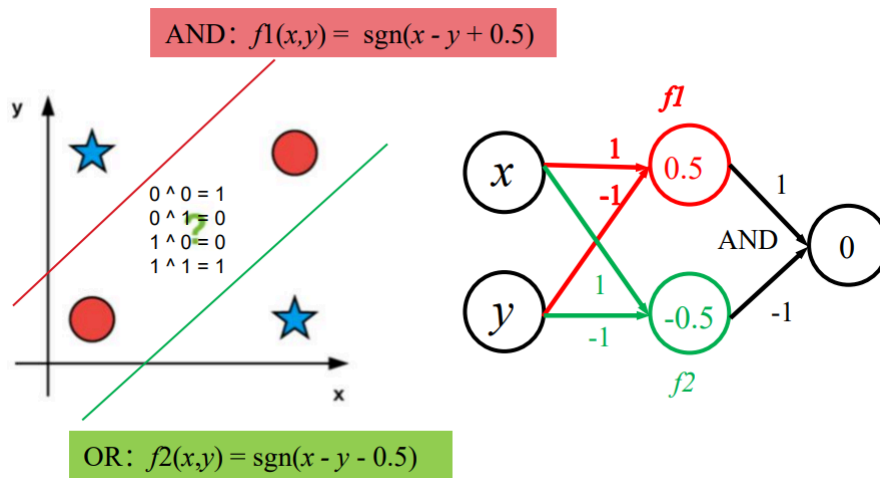
神经网络本质是连续函数的组合。

感知机

一个最简单的二分类线性模型：

$$\text{output} = \text{sign}(w^T x + b) = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

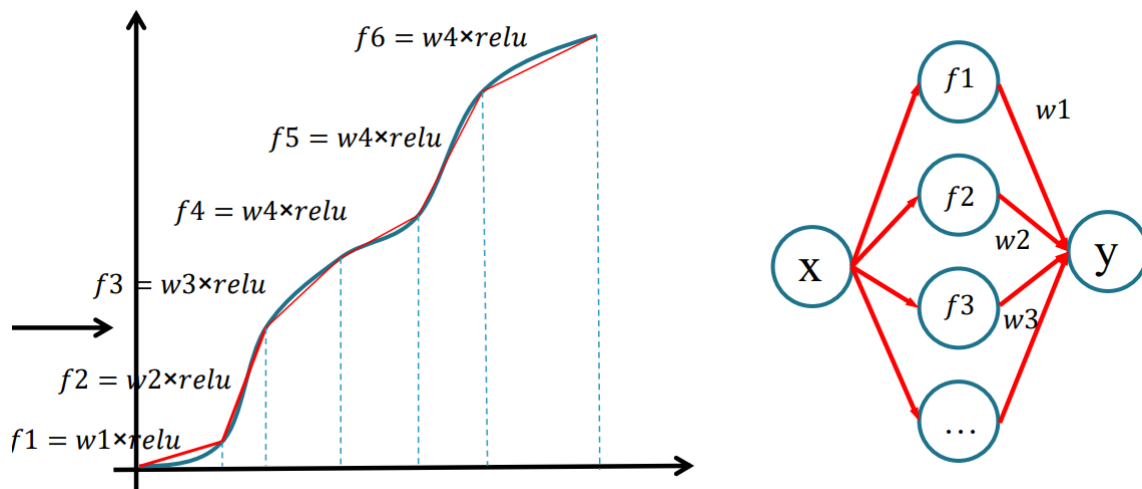




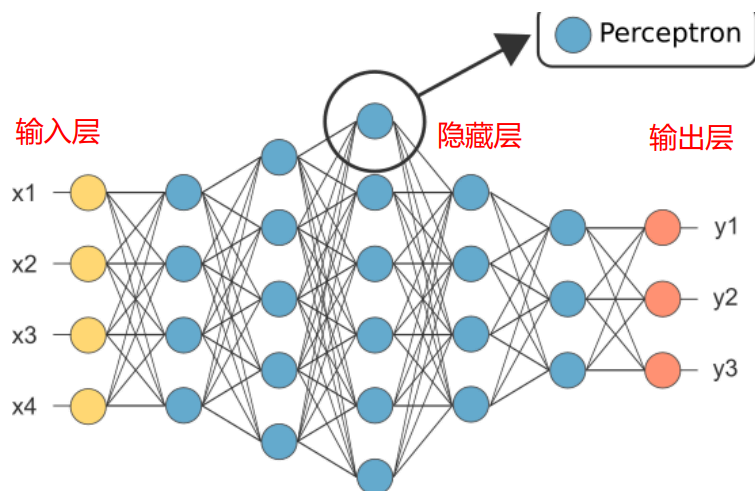
通用逼近定理

对于任意一个定义在紧致集合上的连续函数，只要神经网络具有至少一层隐藏层，并且使用非线性激活函数，就可以以任意精度逼近（非完全等于）该函数。

$$f(x) \approx \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i)$$

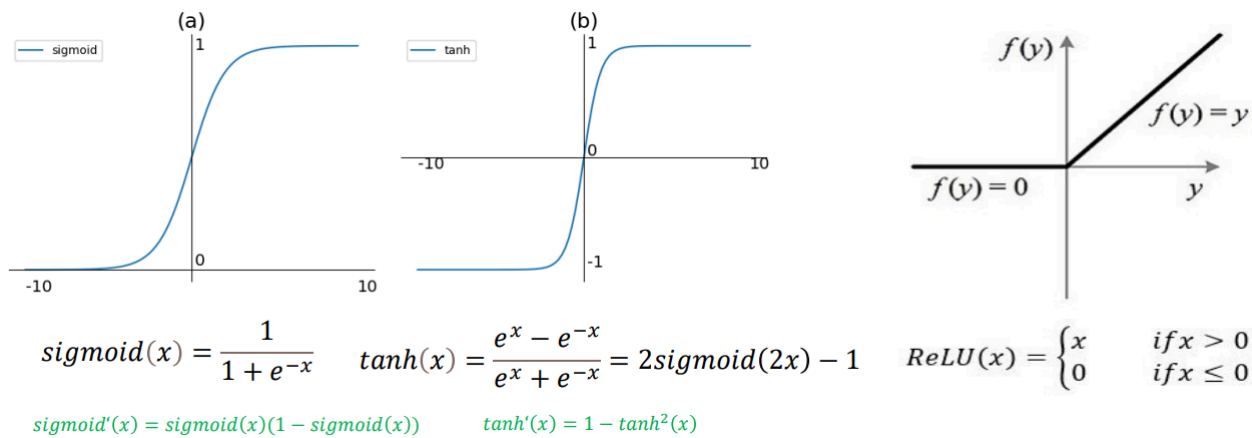


全连接神经网络/多层感知机



1. 复合函数 $f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$
2. 激活函数
3. 梯度下降-反向传播算法

激活函数



激活函数用于引入非线性。

如果不使用激活函数，多层网络在数学上会坍缩回单层线性网络： $y = W_3(W_2(W_1x)) = Wx$ 。

有了非线性激活函数 f ，网络才能具备模拟复杂函数的能力： $y = f_3(W_3f_2(W_2f_1(W_1x)))$ 。

监督学习和损失函数

损失函数：衡量预测值和真实值之间的差距

监督学习通过最小化损失函数，让模型输出逐渐接近真实标签。

输入 → 神经网络（激活函数） → 预测 → 损失函数 → 反向传播 → 更新参数

前向传播

前向传播的过程非常直观，就是数据在网络层之间按顺序传递，最终计算出预测值的过程。

核心步骤

假设我们有一个多层感知机（MLP）：

- 1. 线性组合：每一层的神经元接收上一层的输入 $a^{(l-1)}$ ，通过权重 $W^{(l)}$ 和偏置 $b^{(l)}$ 进行线性变换：

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

- 2. 非线性激活：将得到的 $z^{(l)}$ 通过激活函数（如 ReLU、Sigmoid 或 Tanh），得到该层的输出：

$$a^{(l)} = \sigma(z^{(l)})$$

- 3. 计算损失：当数据到达输出层 L 后，我们会得到预测值 $\hat{y}$ 。接着使用损失函数（Loss Function）来衡量预测值与真实标签 y 之间的差距：

$$a^{(l)} = \sigma(z^{(l)})$$

4.

假设我们有一个最简单的三层全连接网络：

- 输入层：2个神经元（特征 x_1, x_2 ）
- 隐藏层：2个神经元（使用 Sigmoid 激活）
- 输出层：1个神经元（使用Sigmoid 激活，做二分类）

步骤1：隐藏层计算

对于隐藏层第 j 个神经元 ($j = 1, 2$)，其**线性加权和**为：

$$z_j = \sum_{i=1}^2 w_{ij} \cdot x_i + b_j$$

代入激活函数sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ ：

$$h_j = \sigma(z_j)$$

步骤2：输出层计算

输出层神经元的线性加权和：

$$z_0 = w_3 \cdot h_1 + w_4 \cdot h_2 + b_3$$

输出值（预测值）：

$$\hat{y} = \sigma(z_0)$$

这就是一次**前向传播**，网络给出了一个预测值 $\hat{y}$ 。

反向传播（BP）

反向传播的目的是**计算损失函数对每个权重和偏置的梯度**，以便使用梯度下降法更新参数。BP 算法的核心就是微积分中的链式法则（Chain Rule）。

核心逻辑

我们的目标是**最小化损失函数 J** 。为了做到这一点，我们需要知道损失函数对每一个参数（ W 和 b ）的敏感程度，即计算梯度 $\frac{\partial J}{\partial W}$ 和 $\frac{\partial J}{\partial b}$ 。

链式法则的运用

以输出层的一个权重 w 为例，损失对它的梯度可以拆解为：

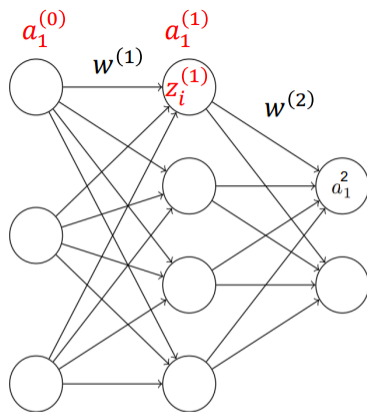
$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- $\frac{\partial J}{\partial a}$ ：误差对输出的影响。
- $\frac{\partial a}{\partial z}$ ：激活函数的导数。
- $\frac{\partial z}{\partial w}$ ：输入值的大小。

BP 的四个核心公式（简记版）

1. **输出层误差**：计算预测值与真实值的偏差。
2. **误差反向传播**：将当前层的误差通过权重矩阵传回前一层。

3. **偏置梯度**：误差直接对应偏置的更新。
4. **权重梯度**：当前层的误差乘以进入该层的输入信号。



$$\frac{\partial c}{\partial a} = \frac{\partial c}{\partial b} \times \frac{\partial b}{\partial a} \qquad \vdots$$

前向传播：

$$z^{(l)} = w^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)}) = f(w^{(l)}a^{(l-1)} + b^{(l)})$$

定义残差：

$$\delta^{(l)} = \frac{\partial J(W,b;x,y)}{\partial z^{(l)}}$$

将损失对“激活前线性值 z ”的偏导数定义为本层误差

损失函数：

$$J(W,b;x,y) = \frac{1}{2}(a^{(L)} - y)^2$$

输出层：

$$\delta^{(L)} = \frac{\partial J(W,b;x,y)}{\partial z^{(L)}} = (a^{(L)} - y) \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}} = (a^{(L)} - y) \cdot f'(z^{(L)})$$

其它层：

$$\delta^{(l)} = \frac{\partial J}{\partial Z^{(l)}} = \frac{\partial J}{\partial Z^{(l+1)}} \frac{\partial Z^{(l+1)}}{\partial Z^{(l)}} = (\delta^{(l+1)}(W^{(l+1)})^T) \cdot f'(z^{(l)})$$

$(\delta^{(l+1)}(W^{(l+1)})^T)$ 这部分是在反向传播误差信号，权重矩阵的转置充当了“反向连接器”。

梯度：

$$\frac{\partial J(W,b;x,y)}{\partial w^{(l)}} = \frac{\partial J}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w^{(l)}} = a^{(l-1)} \cdot \delta^{(l)}$$

符号	含义	维度（形状）约定	备注
上标 (l)	第 l 层	—	$l = 0$ 是输入层， $l = L$ 是输出层
$z^{(l)}$	第 l 层的 线性加权和 （未激活）	$(n_l \times 1)$ 或 $(m \times n_l)$	n_l 是本层神经元个数；若考虑批量则为 $m \times n_l$

符号	含义	维度（形状）约定	备注
$a^{(l)}$	第 l 层的 激活值 （激活后输出）	与 $z^{(l)}$ 相同	$a^{(0)} = x$ （输入特征）
$W^{(l)}$	第 $l - 1$ 层到第 l 层的 权重矩阵	$(n_{l-1} \times n_l)$	行数 = 上层神经元数，列数 = 本层神经元数
$b^{(l)}$	第 l 层的 偏置向量	$(1 \times n_l)$ 或 $(n_l \times 1)$	广播添加到每个样本上
$f(\cdot)$	激活函数	逐元素函数	如 Sigmoid、ReLU、Tanh
$f'(\cdot)$	激活函数的 导数	与输入同维	如 Sigmoid: $f'(z) = a(1 - a)$
J	损失函数 （代价函数）	标量	网络预测与真实标签的误差
L	网络总层数（输出层编号）	—	L 通常指最后一层
y	真实标签 (Ground Truth)	与 $a^{(L)}$ 同维	可以是标量（回归）或向量（分类）
$\delta^{(l)}$	第 l 层的 残差 （误差信号）	与 $z^{(l)}$ 同维	定义为损失对 $z^{(l)}$ 的偏导数
$\odot$	逐元素乘积 (Hadamard Product)	—	两个同维矩阵对应位置相乘
$(W^{(l)})^T$	权重矩阵的 转置	$(n_l \times n_{l-1})$	反向传播时用于将误差从 l 层传回 $l - 1$ 层
$\frac{\partial J}{\partial W^{(l)}}$	损失对权重的 梯度	与 $W^{(l)}$ 同维	用于梯度下降更新参数
$\frac{\partial J}{\partial b^{(l)}}$	损失对偏置的 梯度	与 $b^{(l)}$ 同维	公式中未显式写出，等于 $\delta^{(l)}$ 的列和